DBMS COURSE PROJECT

EXAMINATION SCHEDULING AND RESULT PROCESSING SYSTEM

PROJECT REPORT

Submitted by

Lade Bilvesh Datta

Roll No.: 25WU0102135

B.Tech – Artificial Intelligence & Machine Learning

Woxsen University

Faculty Guide: Dr. Kiran Mayee

2026

Abstract

The Examination Scheduling and Result Processing System is a database-driven web application developed to organize and manage examination-related information in a structured manner. The system maintains records of students, courses, examinations, faculty members, examination halls, marks and results. It also supports hall allocation and invigilator assignment through associative tables.

The application uses Python Flask for the web application layer and MySQL for persistent data storage. A single dashboard provides operations for inserting, updating, deleting and viewing records. Primary keys, foreign keys and composite keys are used to maintain relationships and data integrity. The database design follows normalization principles to reduce redundancy and improve consistency.

The project demonstrates practical DBMS concepts including ER modeling, relational schema design, normalization, SQL, constraints, CRUD operations and database connectivity. Testing confirms that changes made through the interface are reflected in the MySQL database.

1. Introduction

Examination management involves maintaining several related types of information such as student details, courses, examination schedules, faculty assignments, examination halls, marks and results. Managing these records manually or in disconnected files can lead to duplication, inconsistency and difficulty in retrieving information.

The Examination Scheduling and Result Processing System provides a centralized relational database and web interface for handling these activities. The system organizes related information into separate tables and connects them through primary and foreign keys.

The project was developed as a practical implementation of database management concepts. It combines a Flask-based user interface with a MySQL relational database so that users can perform common database operations through one dashboard.

2. Problem Statement

Educational institutions need to manage a large amount of examination information, including student records, courses, examination dates, faculty members, examination halls, invigilator assignments, marks and results. When such information is maintained manually or across separate files, it becomes difficult to keep the data consistent and retrieve accurate information efficiently.

The absence of a centralized database can result in duplicate records, incorrect associations between examinations and halls or faculty, and difficulties in maintaining marks and result information. Examination scheduling and result processing therefore require a structured system that can store related information and maintain relationships between records.

The proposed system addresses this problem by implementing a relational database with a web-based interface. It provides CRUD operations and uses database constraints and relationships to maintain organized and consistent examination information.

3. Objectives

- Design a structured relational database for examination scheduling and result processing.
- Maintain accurate information about students, courses, examinations, faculty and examination halls.
- Manage hall allocation and invigilator assignment through related database records.
- Store marks and result information in an organized and retrievable form.
- Support insertion, updating, deletion and viewing operations through a web-based interface.
- Maintain data consistency using primary keys, foreign keys and appropriate composite keys.
- Reduce data redundancy and improve organization through a structured relational database design.
- Demonstrate practical DBMS concepts such as entity relationships, normalization, constraints, relational schema design and database connectivity.

4. Scope of the Project

The scope of the project covers the design and implementation of a database system for managing examinationrelated information. The system maintains records of students, courses, examinations, faculty members, examination halls, marks and results.

The system also supports examination scheduling, hall allocation and invigilator assignment. Through the webbased interface, users can perform operations such as inserting, updating, deleting and viewing records. The system connects the frontend interface with the MySQL database so that changes made through the application are reflected in the database.

The project focuses on demonstrating practical DBMS concepts such as entity relationships, normalization, constraints, relational schema design and database connectivity.

5. Functional Requirements

- Student Management – Add, update, delete and view student records.
- Course Management – Store and manage course information including course name and credits.
- Examination Management – Schedule examinations by associating courses with examination dates.
- Faculty Management – Maintain faculty information for examination-related activities.
- Exam Hall Management – Store examination hall details and seating capacity.
- Hall Allocation – Allocate examination halls to scheduled examinations.
- Invigilator Assignment – Assign faculty members as invigilators for examinations.
- Marks Management – Enter, update and view marks obtained by students.
- Result Management – Store and display student results, grades and status.
- Database Connectivity – Connect the web application with MySQL for storing and retrieving information.

6. Software and Hardware Requirements

6.1 Software Requirements

- Operating System: Windows 10/11
- Programming Language: Python
- Web Framework: Flask
- Database Management System: MySQL
- Database Connector: MySQL Connector/Python
- Frontend: HTML and CSS
- Development Environment: Visual Studio Code
- Web Browser: Google Chrome / Microsoft Edge
- Version Control: Git and GitHub

6.2 Hardware Requirements

- Processor: Intel Core i3 or equivalent and above
- RAM: Minimum 4 GB
- Storage: Minimum 10 GB free space
- Display: Standard monitor
- Network: Internet connection for GitHub and software installation

7. System Architecture and Implementation

The system follows a simple three-layer architecture consisting of the presentation layer, application layer and database layer.

- Presentation Layer: Provides the web-based dashboard and forms used to interact with the system.
- Application Layer: Implemented using Python and Flask; handles routing, form processing, CRUD operations and database communication.
- Database Layer: Implemented using MySQL; stores and manages all examination-related records.

The Flask application establishes a connection with the MySQL database and executes SQL queries based on user actions. This allows insertion, updating, deletion and retrieval of records through a single dashboard.

8. Entity–Relationship Model

The Entity–Relationship model represents the major entities involved in the examination scheduling and result processing system and the relationships between them. The main entities are STUDENT, COURSE, EXAMINATION, FACULTY, EXAM_HALL, MARKS and RESULT. The associative entities EXAM_INVIGILATOR and HALL_ALLOCATION are used to handle many-to-many relationships.

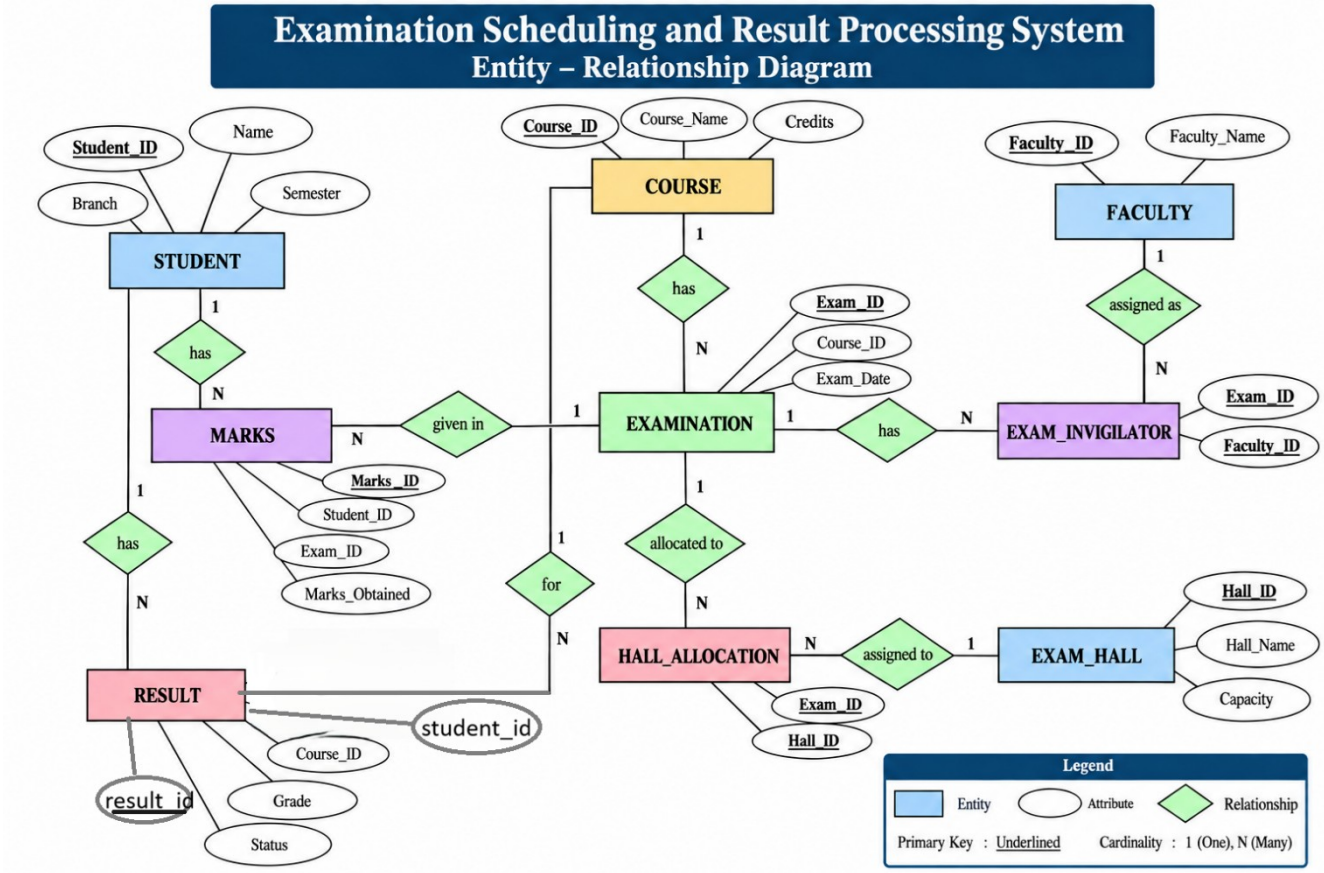


Figure 8.1: Entity–Relationship Diagram of the System

8.1 Relationships and Cardinality

- COURSE 1:N EXAMINATION
- STUDENT 1:N MARKS
- EXAMINATION 1:N MARKS
- STUDENT 1:N RESULT

- COURSE 1:N RESULT
- EXAMINATION 1:N EXAM_INVIGILATOR
- FACULTY 1:N EXAM_INVIGILATOR
- EXAMINATION 1:N HALL_ALLOCATION
- EXAM_HALL 1:N HALL_ALLOCATION
- EXAMINATION M:N FACULTY through EXAM_INVIGILATOR
- EXAMINATION M:N EXAM_HALL through HALL_ALLOCATION

9. Relational Schema

The relational schema consists of nine tables. Primary keys uniquely identify records, while foreign keys establish relationships between related tables.

- STUDENT(Student_ID PK, Name, Branch, Semester)
- COURSE(Course_ID PK, Course_Name, Credits)
- EXAMINATION(Exam_ID PK, Course_ID FK, Exam_Date)
- FACULTY(Faculty_ID PK, Faculty_Name)
- EXAM_HALL(Hall_ID PK, Hall_Name, Capacity)
- MARKS(Marks_ID PK, Student_ID FK, Exam_ID FK, Marks_Obtained)
- RESULT(Result_ID PK, Student_ID FK, Course_ID FK, Grade, Status)
- EXAM_INVIGILATOR(Exam_ID PK/FK, Faculty_ID PK/FK)
- HALL_ALLOCATION(Exam_ID PK/FK, Hall_ID PK/FK)

10. Normalization

The database is designed using normalization principles to reduce redundancy and maintain data consistency. The tables are structured so that each attribute contains a single value and repeating groups are avoided.

The design follows the principles of First Normal Form (1NF), Second Normal Form (2NF) and Third Normal Form (3NF). Separate tables for students, courses, examinations, faculty and halls prevent unnecessary duplication. Associative tables are used for many-to-many relationships between examinations and faculty members and between examinations and examination halls.

The resulting design is intended to be in Third Normal Form because non-key attributes depend on the key of their respective relation and transitive dependencies are avoided in the main tables.

11. Data Dictionary

Table	Attribute	Description
STUDENT	Student_ID	Unique identifier of a student
STUDENT	Name	Name of the student
STUDENT	Branch	Academic branch of the student
STUDENT	Semester	Current semester of the student
COURSE	Course_ID	Unique identifier of a course
COURSE	Course_Name	Name of the course
COURSE	Credits	Credits assigned to the course

EXAMINATION	Exam_ID	Unique examination identifier
EXAMINATION	Course_ID	Course associated with the examination
EXAMINATION	Exam_Date	Date of the examination
FACULTY	Faculty_ID	Unique faculty identifier
FACULTY	Faculty_Name	Name of the faculty member
EXAM_HALL	Hall_ID	Unique examination hall identifier
EXAM_HALL	Hall_Name	Name of the examination hall
EXAM_HALL	Capacity	Maximum capacity of the hall
MARKS	Marks_ID	Unique marks record identifier
MARKS	Student_ID	Student associated with the marks
MARKS	Exam_ID	Examination associated with the marks
MARKS	Marks_Obtained	Marks obtained by the student
RESULT	Result_ID	Unique result identifier
RESULT	Student_ID	Student associated with the result
RESULT	Course_ID	Course associated with the result
RESULT	Grade	Grade obtained by the student
RESULT	Status	Result status
EXAM_INVIGILATOR	Exam_ID	Examination assigned to the faculty
EXAM_INVIGILATOR	Faculty_ID	Faculty member assigned as invigilator
HALL_ALLOCATION	Exam_ID	Examination assigned to the hall
HALL_ALLOCATION	Hall_ID	Hall allocated to the examination

12. SQL Commands

The following SQL commands illustrate the database creation and table structure used by the project. The application performs CRUD operations on these relations through Flask.

```
CREATE DATABASE examination_db;
USE examination_db;

CREATE TABLE STUDENT (
   Student_ID INT PRIMARY KEY,
   Name VARCHAR(100) NOT NULL,
   Branch VARCHAR(100),
   Semester INT
);

CREATE TABLE COURSE (
   Course_ID INT PRIMARY KEY,
   Course_Name VARCHAR(100) NOT NULL,
   Credits INT
);

CREATE TABLE EXAMINATION (
   Exam_ID INT PRIMARY KEY,
```

```
    Course_ID INT,
    Exam_Date DATE,
    FOREIGN KEY (Course_ID) REFERENCES COURSE(Course_ID)
);

CREATE TABLE FACULTY (
    Faculty_ID INT PRIMARY KEY,
    Faculty_Name VARCHAR(100) NOT NULL
);

CREATE TABLE EXAM_HALL (
    Hall_ID INT PRIMARY KEY,
    Hall_Name VARCHAR(100) NOT NULL,
    Capacity INT
);

CREATE TABLE MARKS (
    Marks_ID INT PRIMARY KEY,
    Student_ID INT,
    Exam_ID INT,
    Marks_Obtained INT,
    FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
    FOREIGN KEY (Exam_ID) REFERENCES EXAMINATION(Exam_ID)
);

CREATE TABLE RESULT (
    Result_ID INT PRIMARY KEY,
    Student_ID INT,
    Course_ID INT,
    Grade VARCHAR(10),     Status
VARCHAR(20),
    FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
    FOREIGN KEY (Course_ID) REFERENCES COURSE(Course_ID)
);

CREATE TABLE EXAM_INVIGILATOR (
    Exam_ID INT,
    Faculty_ID INT,
    PRIMARY KEY (Exam_ID, Faculty_ID),
    FOREIGN KEY (Exam_ID) REFERENCES EXAMINATION(Exam_ID),
    FOREIGN KEY (Faculty_ID) REFERENCES FACULTY(Faculty_ID)
);

CREATE TABLE HALL_ALLOCATION (
    Exam_ID INT,
    Hall_ID INT,
    PRIMARY KEY (Exam_ID, Hall_ID),
    FOREIGN KEY (Exam_ID) REFERENCES EXAMINATION(Exam_ID),
    FOREIGN KEY (Hall_ID) REFERENCES EXAM_HALL(Hall_ID)
);
```

13. SQL Queries

View students: SELECT * FROM STUDENT;

View courses: SELECT * FROM COURSE;

View examinations: SELECT * FROM EXAMINATION;

View marks: SELECT * FROM MARKS;

View results: SELECT * FROM RESULT;

Update student: UPDATE STUDENT SET Name='Updated Name' WHERE Student_ID=1;

Delete student: DELETE FROM STUDENT WHERE Student_ID=1;

Join examination with course: SELECT E.Exam_ID, C.Course_Name, E.Exam_Date FROM EXAMINATION E JOIN COURSE C ON E.Course_ID=C.Course_ID;

14. User Interface Design

The system provides a single web-based dashboard for managing examination-related activities. Navigation options are provided for students, courses, examinations, faculty, examination halls, hall allocation, invigilators, marks and results.

Each section contains forms and tables for database operations. Users can add new records, update existing records, delete records and view stored information. The interface communicates with MySQL through the Flask application.

15. Implementation Details

The application is implemented using Python Flask as the backend framework and MySQL as the database management system. HTML and CSS are used to create the web interface, while MySQL Connector/Python provides database connectivity.

- Flask routes handle page navigation and CRUD operations.
- HTML forms collect data from the user.
- MySQL queries insert, update, delete and retrieve records.
- Primary and foreign keys maintain relationships between tables.
- Composite keys are used in EXAM_INVIGILATOR and HALL_ALLOCATION.
- The system is operated through a single dashboard.

16. User Interface Screenshots

The following screenshots document the implemented interface and demonstrate the major database operations. The screenshots include initial views, record entry, updated views and management interfaces.

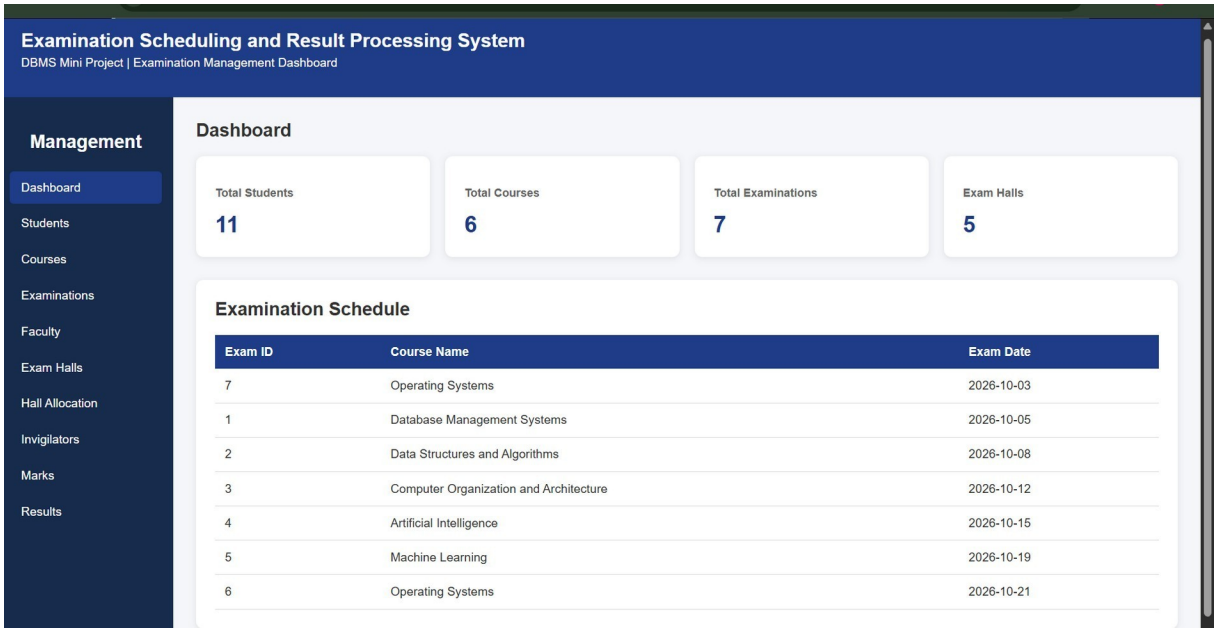


Figure 16.1: Main Dashboard

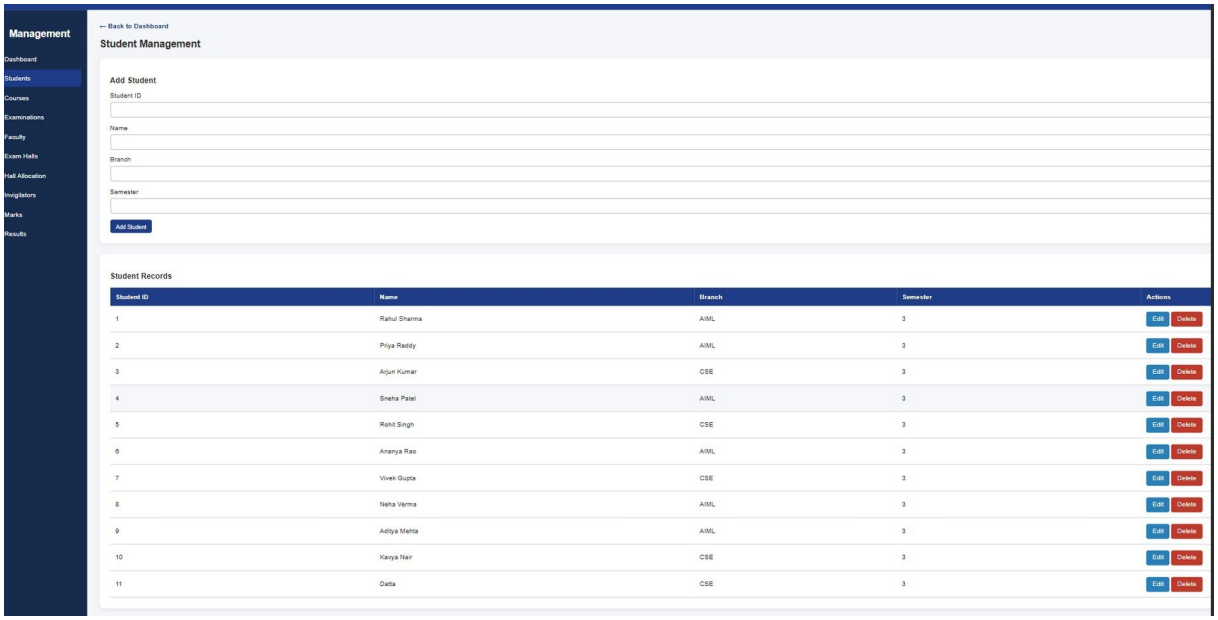


Figure 16.2: Student Management – Initial View

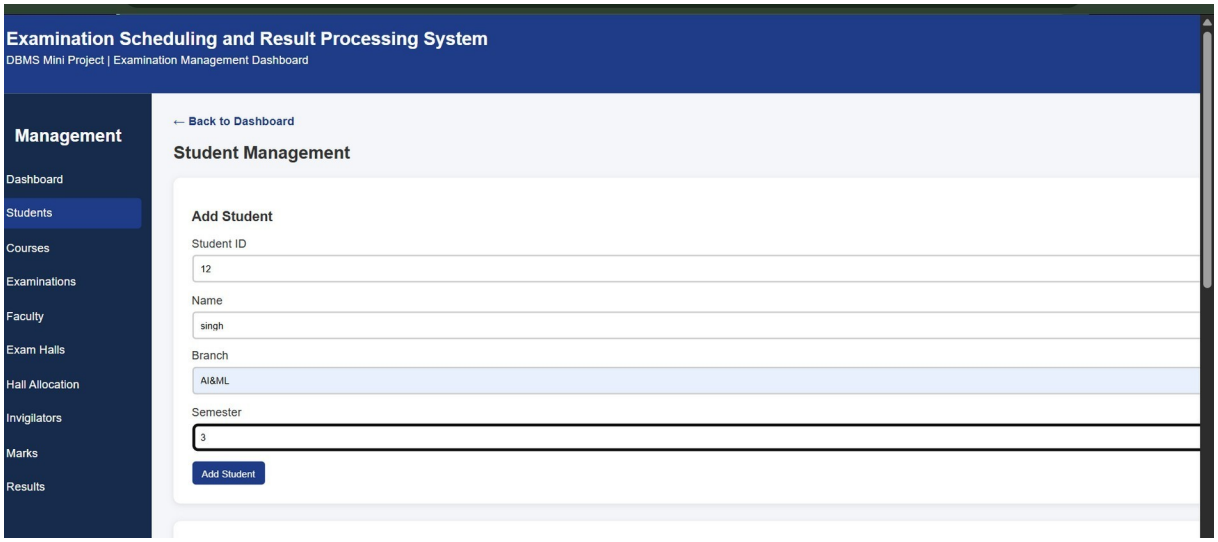


Figure 16.3: Student Entry – New Record

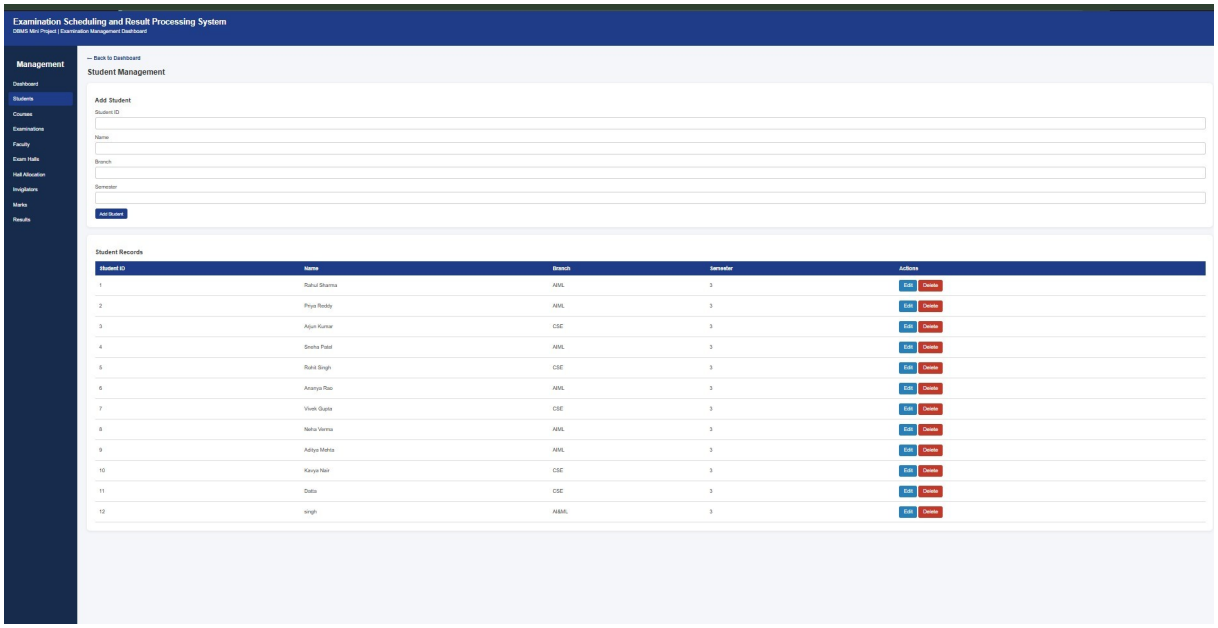


Figure 16.4: Student Management – Updated View

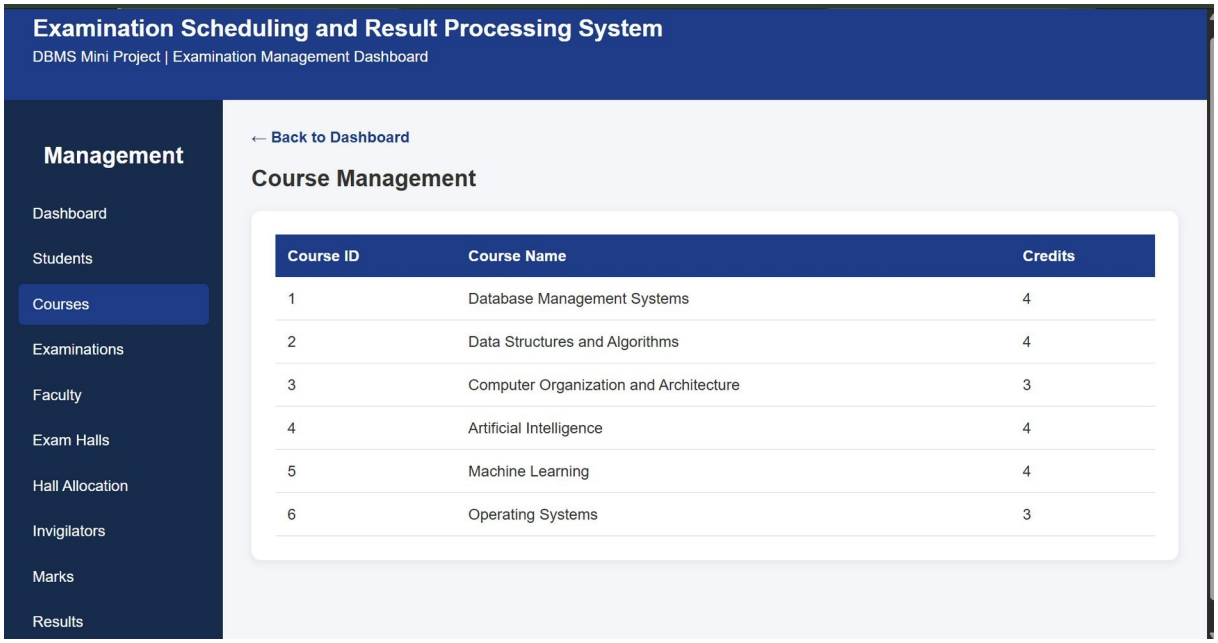


Figure 16.5: Course Management

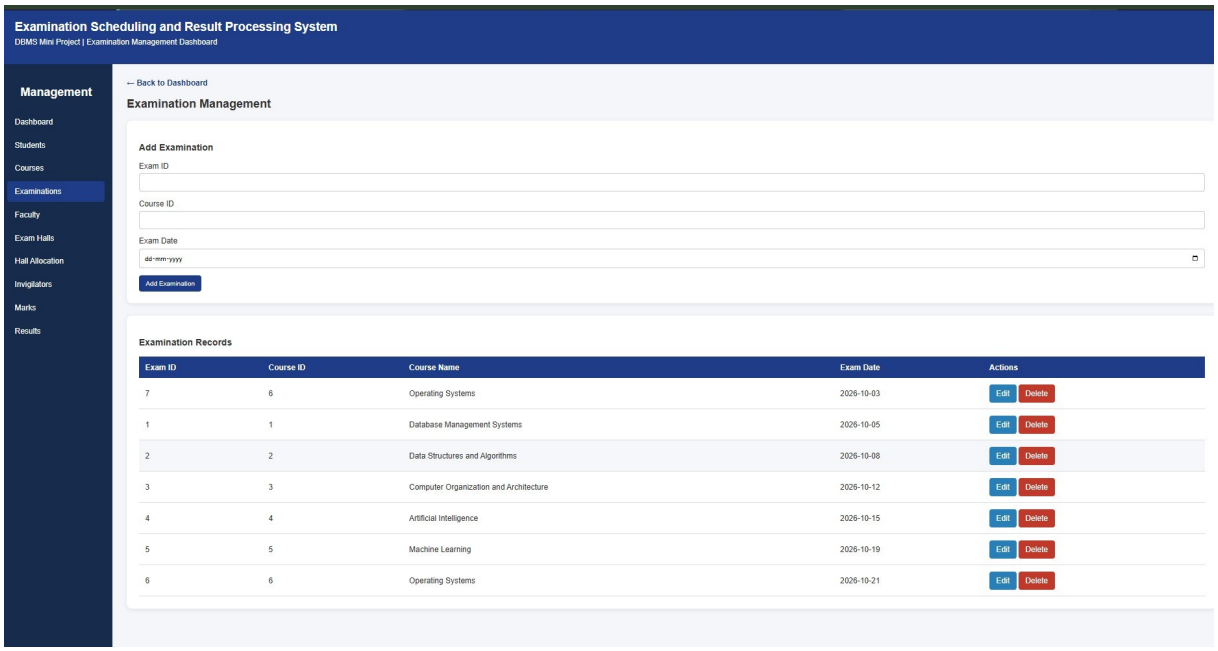


Figure 16.6: Examination Management – Initial View

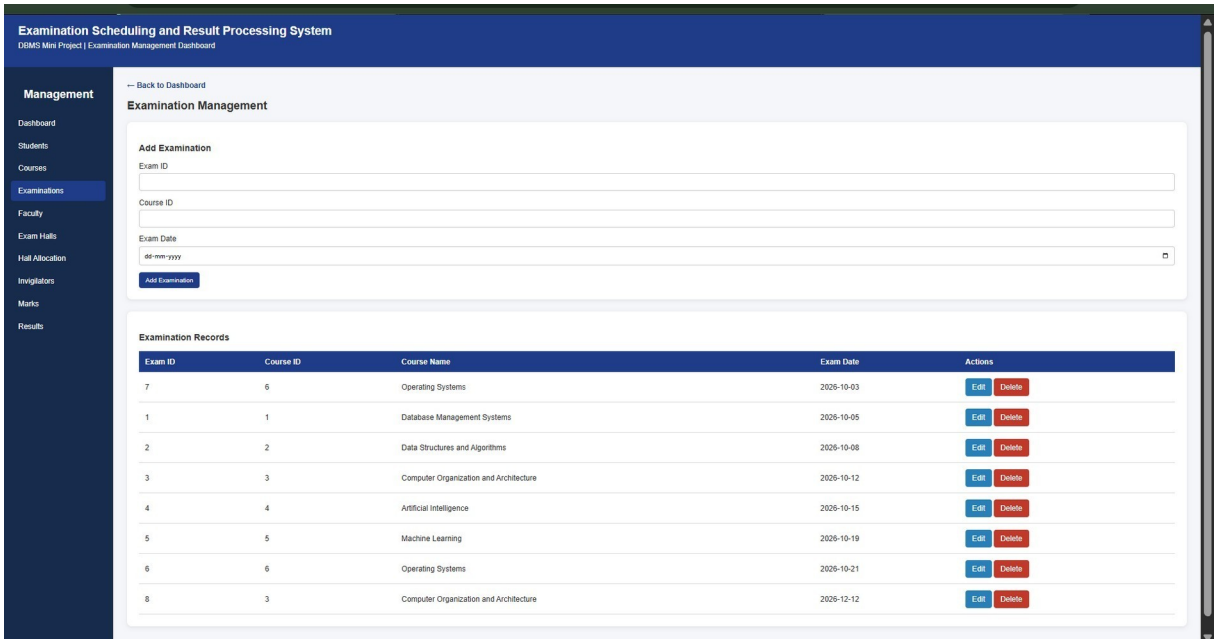


Figure 16.7: Examination Management – Updated View

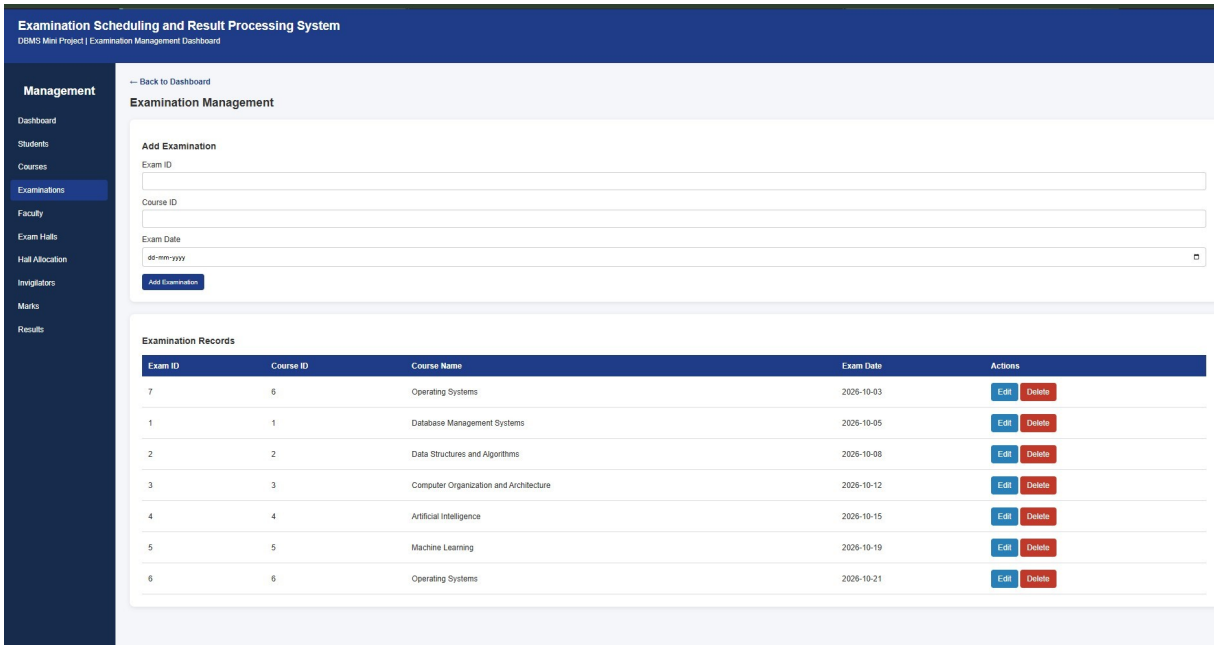


Figure 16.8: Examination Management – Record View

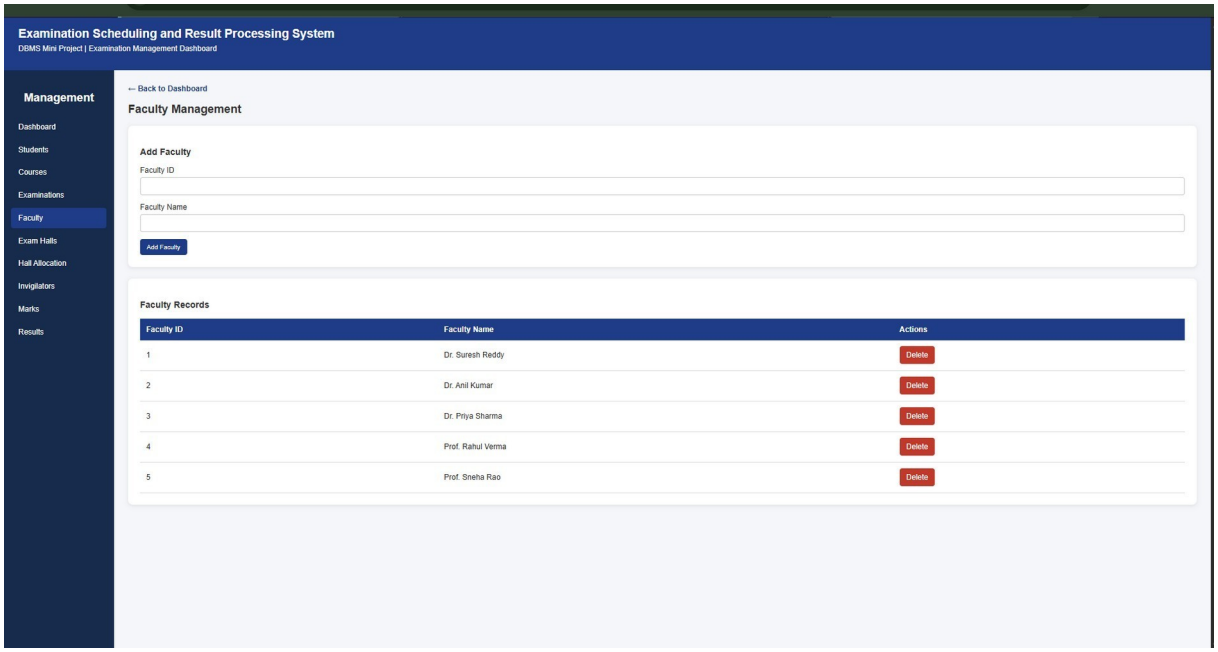


Figure 16.9: Faculty Management – Initial View

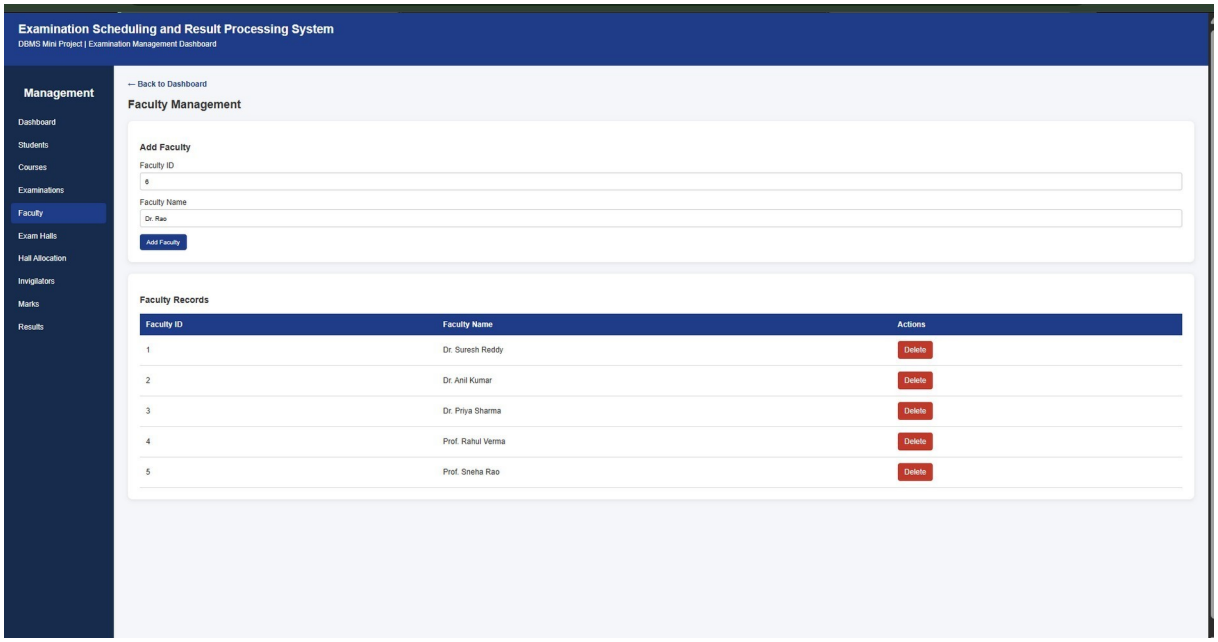


Figure 16.10: Faculty Entry – New Record

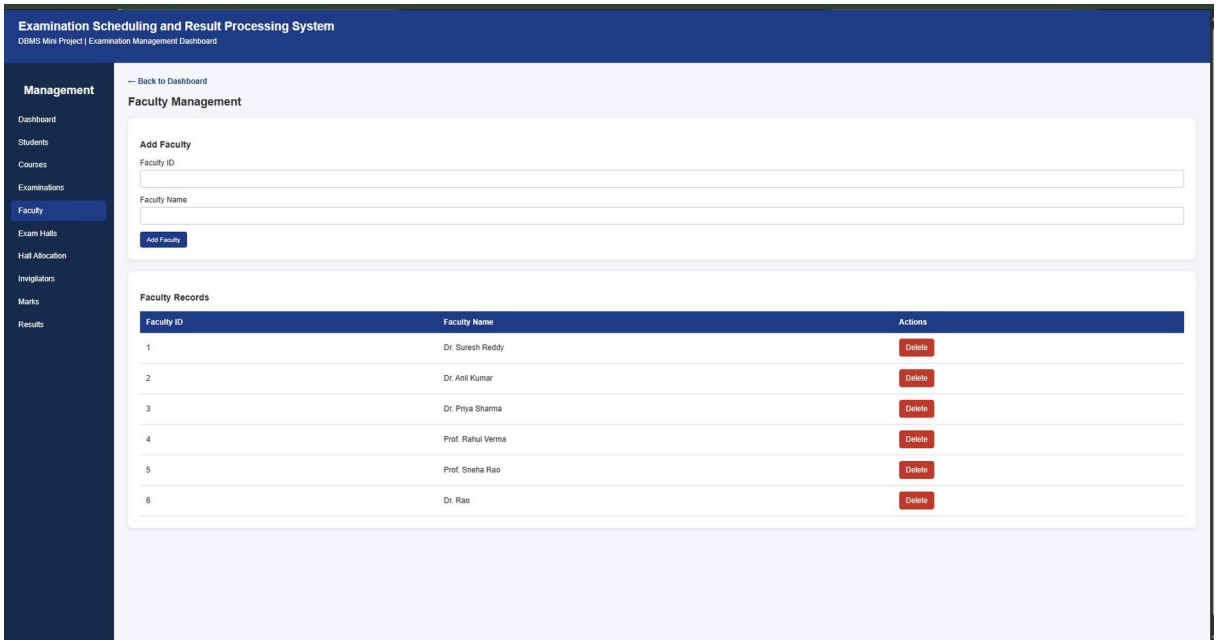


Figure 16.11: Faculty Management – Updated View

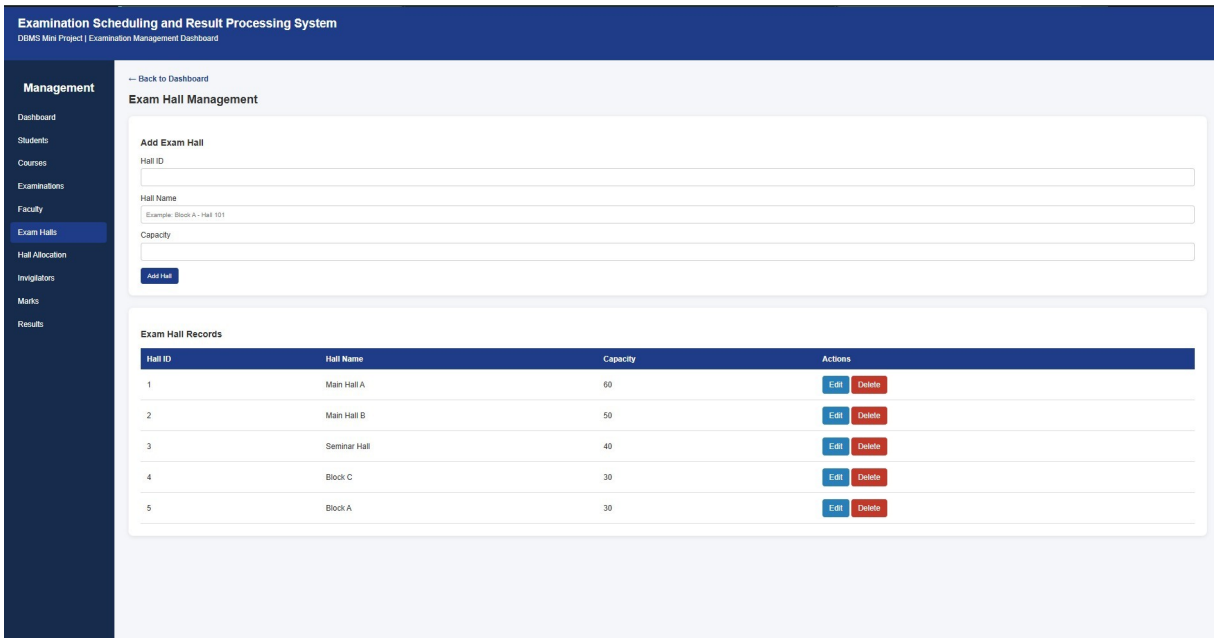


Figure 16.12: Exam Hall Management – Initial View

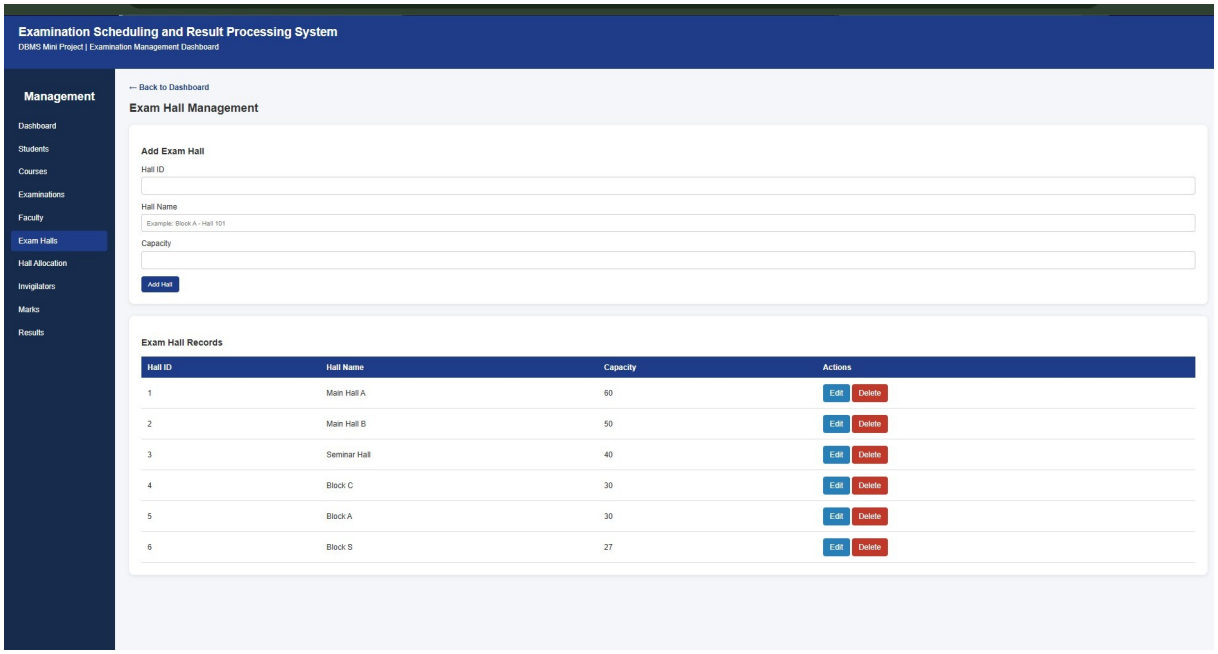


Figure 16.13: Exam Hall Management – Updated View

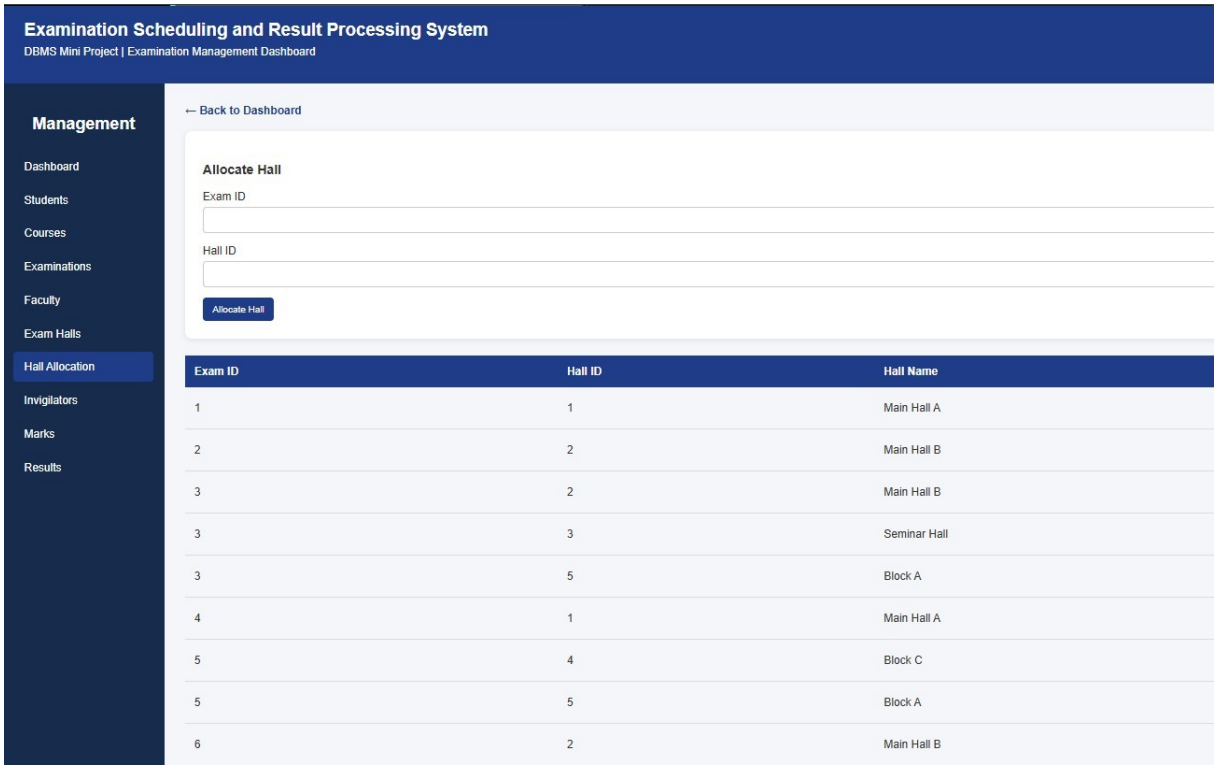


Figure 16.14: Hall Allocation

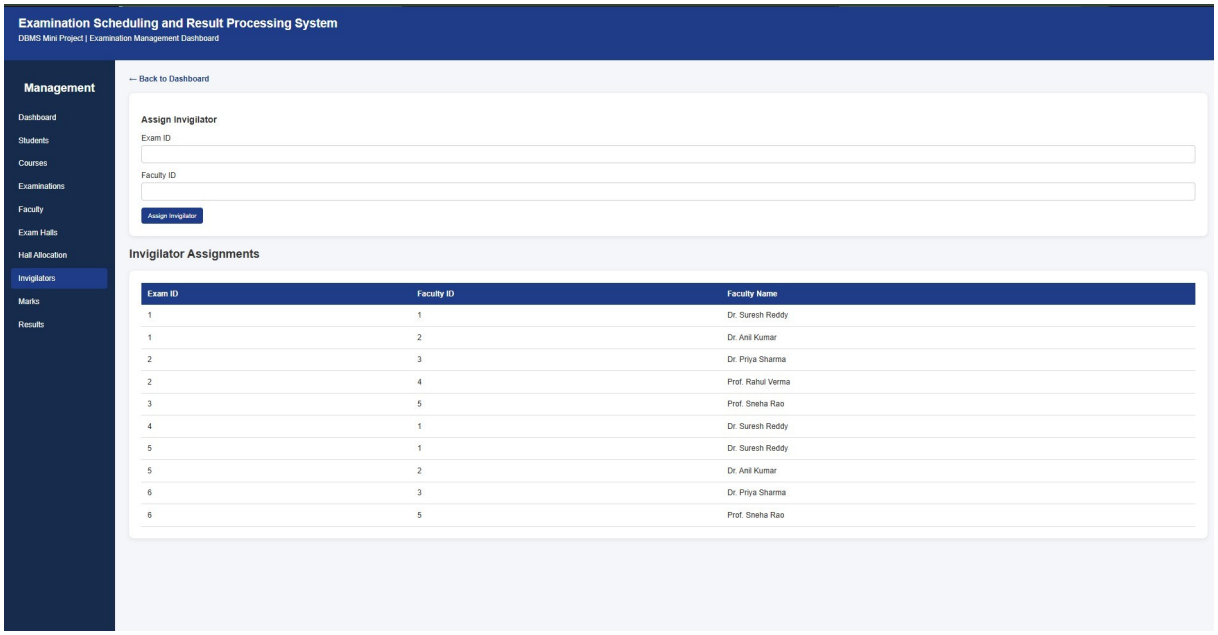


Figure 16.15: Invigilator Assignment Interface

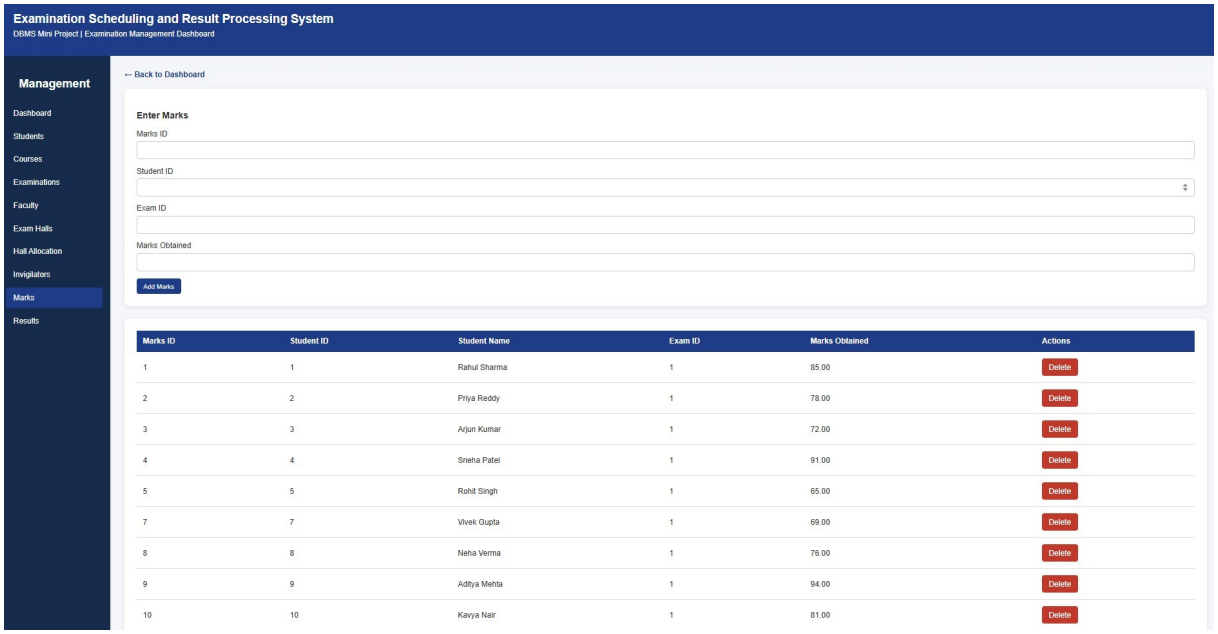


Figure 16.16: Marks Entry and Marks List

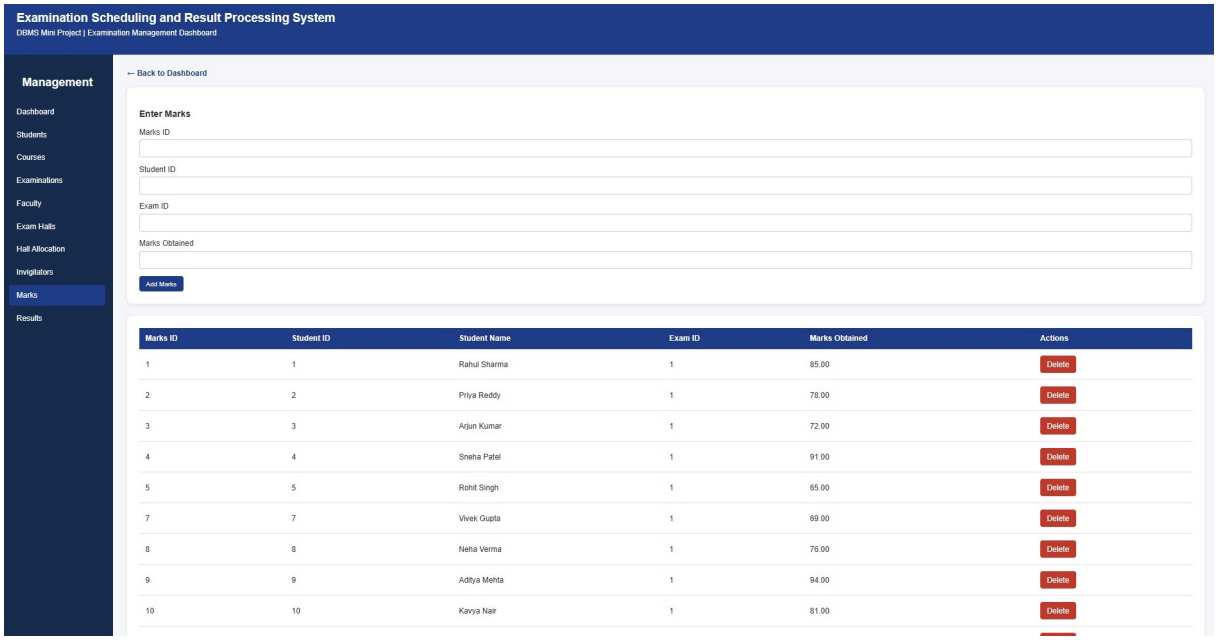


Figure 16.17: Marks List – Updated View

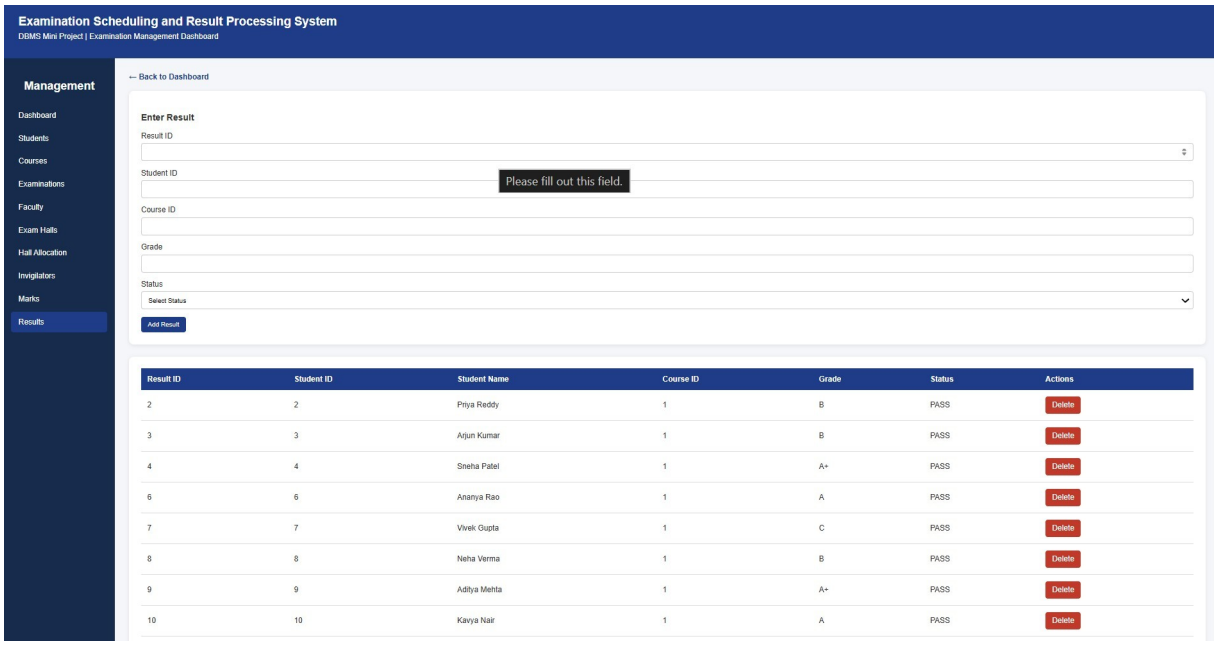


Figure 16.18: Result Entry and Result List

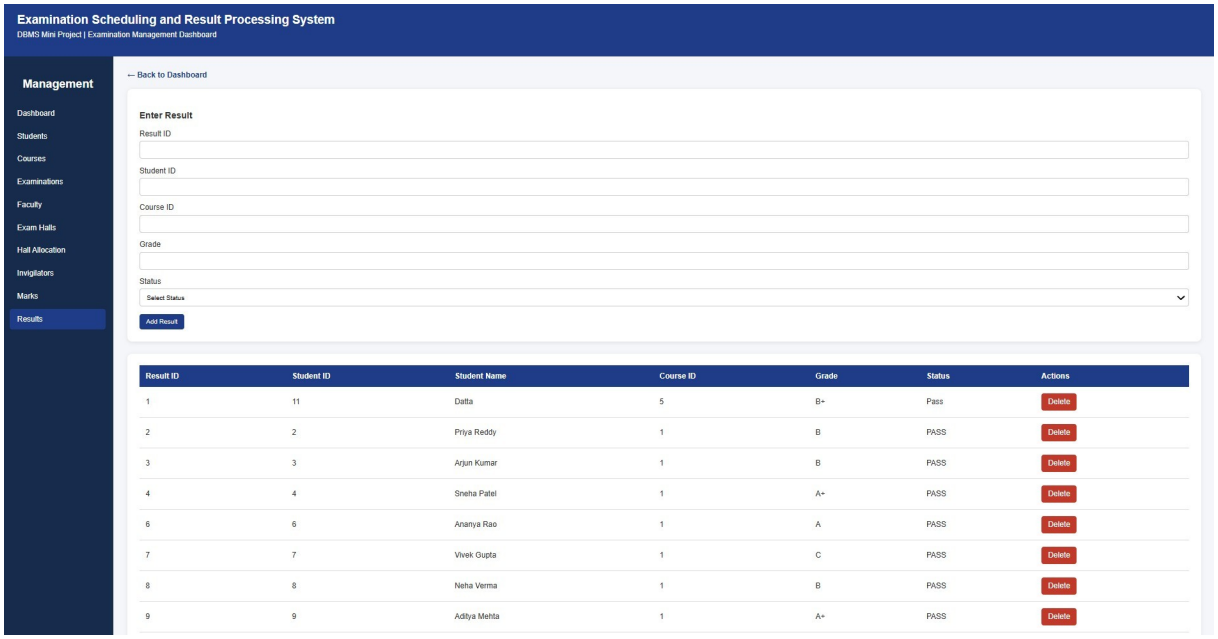


Figure 16.19: Result List – Updated View

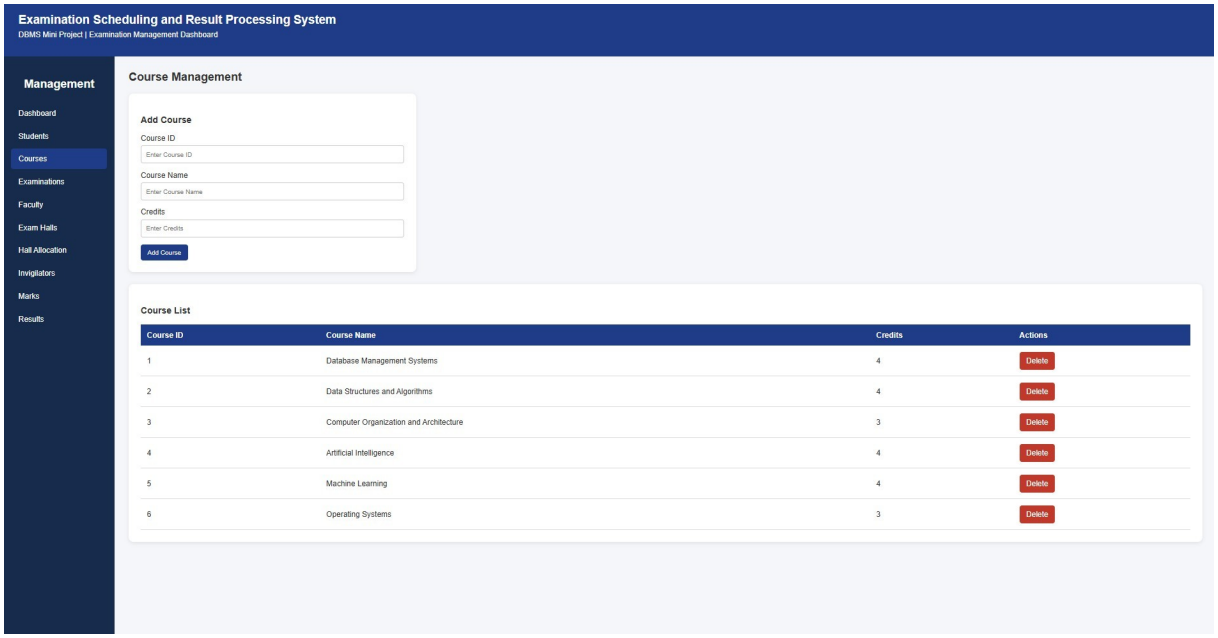


Figure 16.20: Course Management – Additional View

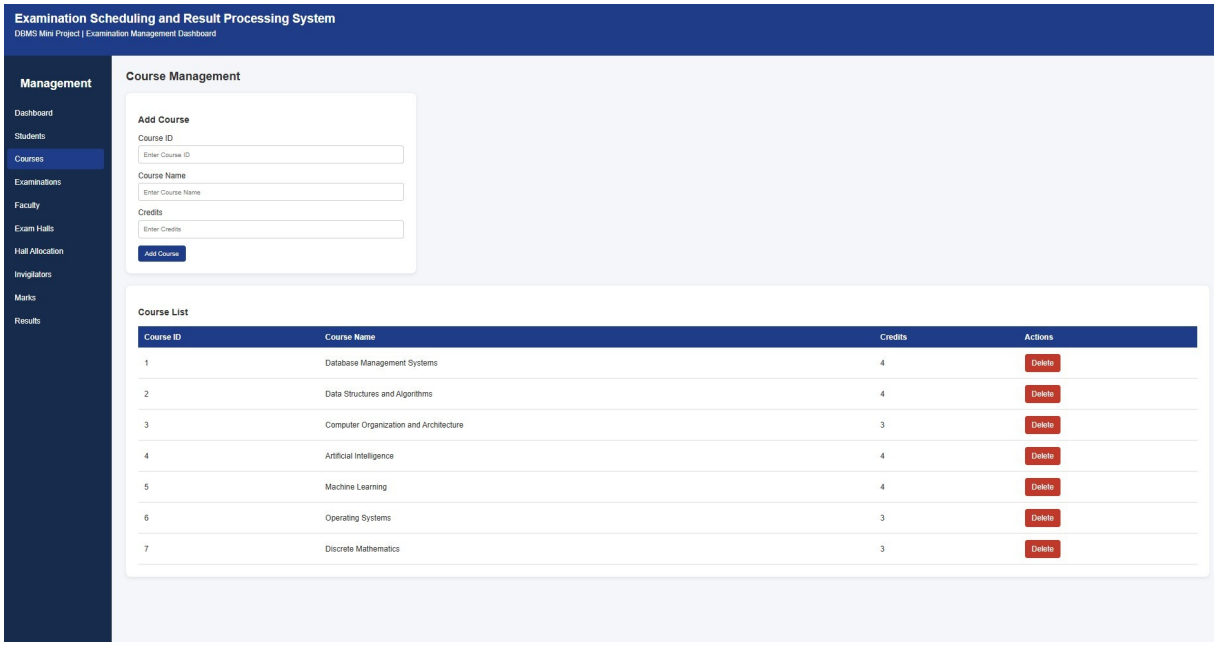


Figure 16.21: Course Management – Updated View

17. Testing

Test Case	Operation	Expected Result	Status
TC01	Add Student	New student record is inserted	Passed
TC02	Update Student	Student details are updated	Passed
TC03	Delete Student	Student record is deleted	Passed
TC04	Add Examination	Examination record is inserted	Passed
TC05	Update Examination	Examination details are updated	Passed
TC06	Add Faculty	Faculty record is inserted	Passed
TC07	Add Exam Hall	Hall record is inserted	Passed
TC08	Hall Allocation	Hall is allocated to examination	Passed
TC09	Invigilator Assignment	Faculty is assigned to examination	Passed
TC10	Marks Entry	Marks are stored successfully	Passed
TC11	Result Entry	Result is stored successfully	Passed
TC12	View Records	Updated database records are displayed correctly	Passed

18. Conclusion

The Examination Scheduling and Result Processing System successfully implements a database-driven solution for managing examination-related information. The system provides a single web-based dashboard for managing students, courses, examinations, faculty, examination halls, hall allocation, invigilators, marks and results.

The project demonstrates important DBMS concepts such as relational schema design, ER modeling, normalization, primary keys, foreign keys, composite keys and CRUD operations. The integration of Flask with MySQL allows users to perform database operations through a simple and organized interface.

Overall, the project provides a functional database solution for organizing examination scheduling and result processing activities and demonstrates the practical application of database management concepts.

19. Future Enhancements

- User authentication and role-based access.
- Automatic examination timetable generation.
- Automatic hall allocation based on hall capacity.
- Automatic grade calculation from marks.
- PDF generation for examination schedules and results.
- Email or notification facilities.
- Advanced search and filtering options.
- Graphical reports and dashboard statistics.
- Cloud deployment and online database backup.

20. References

1. MySQL 8.4 Reference Manual – https://dev.mysql.com/doc/refman/8.4/en/
2. MySQL SQL Statements – https://dev.mysql.com/doc/refman/8.4/en/sql-statements.html
3. Python Documentation – https://docs.python.org/3/
4. Flask Documentation – https://flask.palletsprojects.com/en/stable/
5. Flask Quickstart – https://flask.palletsprojects.com/en/stable/quickstart/
6. MDN HTML Documentation – https://developer.mozilla.org/en-US/docs/Web/HTML
7. MDN CSS Documentation – https://developer.mozilla.org/en-US/docs/Web/CSS

21. Appendix

Appendix A – Project Database Tables

- STUDENT(Student_ID PK, Name, Branch, Semester)
- COURSE(Course_ID PK, Course_Name, Credits)
- EXAMINATION(Exam_ID PK, Course_ID FK, Exam_Date)
- FACULTY(Faculty_ID PK, Faculty_Name)
- EXAM_HALL(Hall_ID PK, Hall_Name, Capacity)
- MARKS(Marks_ID PK, Student_ID FK, Exam_ID FK, Marks_Obtained)
- RESULT(Result_ID PK, Student_ID FK, Course_ID FK, Grade, Status)

- EXAM_INVIGILATOR(Exam_ID PK/FK, Faculty_ID PK/FK)
- HALL_ALLOCATION(Exam_ID PK/FK, Hall_ID PK/FK)

Appendix B – Project Technologies

- Python
- Flask
- MySQL
- HTML
- CSS
- MySQL Connector/Python
- GitHub

Appendix C – Project Source Code

The project source code contains the Flask application, HTML template, database SQL script and supporting project files. The source code is maintained in the public GitHub repository named DBMS-Course-Project.

Appendix D – Additional Interface Evidence

The interface screenshots included in Section 16 provide visual evidence of insertion, updating, deletion and viewing operations performed through the system.

Appendix E – SQL Query Screenshots and Outputs

The following screenshots show SQL queries executed against the examination_db database and the corresponding results returned by MySQL. They provide evidence of record retrieval and joins across the student, course, examination, marks, hall allocation and invigilator tables.

```
mysql> SELECT * FROM STUDENT;
+------------+--------------+--------+----------+
| Student_ID | Name         | Branch | Semester |
+------------+--------------+--------+----------+
|          1 | Rahul Sharma | AIML   |        3 |
|          2 | Priya Reddy  | AIML   |        3 |
|          3 | Arjun Kumar  | CSE    |        3 |
|          4 | Sneha Patel  | AIML   |        3 |
|          5 | Rohit Singh  | CSE    |        3 |
|          6 | Ananya Rao   | AIML   |        3 |
|          7 | Vivek Gupta  | CSE    |        3 |
|          8 | Neha Verma   | AIML   |        3 |
|          9 | Aditya Mehta | AIML   |        3 |
|         10 | Kavya Nair   | CSE    |        3 |
|         11 | Datta        | CSE    |        2 |
+------------+--------------+--------+----------+
11 rows in set (0.011 sec)
```

Figure E.1: Student Records Retrieved Using SELECT

```
mysql> SELECT
    ->     E.Exam_ID,
    ->     C.Course_Name,
    ->     E.Exam_Date
    -> FROM EXAMINATION E
    -> JOIN COURSE C
    ->     ON E.Course_ID = C.Course_ID;
+---------+---------------------------------------+------------+
| Exam_ID | Course_Name                           | Exam_Date  |
+---------+---------------------------------------+------------+
|       1 | Database Management Systems           | 2026-10-05 |
|       2 | Data Structures and Algorithms        | 2026-10-08 |
|       3 | Computer Organization and Architecture | 2026-10-12 |
|       4 | Artificial Intelligence               | 2026-10-15 |
|       5 | Machine Learning                      | 2026-10-19 |
|       6 | Operating Systems                     | 2026-10-21 |
|       7 | Operating Systems                     | 2026-10-03 |
+---------+---------------------------------------+------------+
7 rows in set (0.130 sec)
```

Figure E.2: Examination Schedule Retrieved Using JOIN

```
mysql> SELECT
    ->     M.Marks_ID,
    ->     S.Name,
    ->     M.Exam_ID,
    ->     M.Marks_Obtained
    -> FROM MARKS M
    -> JOIN STUDENT S
    ->     ON M.Student_ID = S.Student_ID;
+----------+--------------+---------+----------------+
| Marks_ID | Name         | Exam_ID | Marks_Obtained |
+----------+--------------+---------+----------------+
|        1 | Rahul Sharma |       1 |          85.00 |
|       11 | Rahul Sharma |       2 |          79.00 |
|        2 | Priya Reddy  |       1 |          78.00 |
|       12 | Priya Reddy  |       2 |          88.00 |
|        3 | Arjun Kumar  |       1 |          72.00 |
|       13 | Arjun Kumar  |       2 |          67.00 |
|       17 | Arjun Kumar  |       3 |          12.00 |
|        4 | Sneha Patel  |       1 |          91.00 |
|        5 | Rohit Singh  |       1 |          65.00 |
|       15 | Rohit Singh  |       2 |          74.00 |
|       16 | Ananya Rao   |       2 |          83.00 |
|        7 | Vivek Gupta  |       1 |          69.00 |
|        8 | Neha Verma   |       1 |          76.00 |
|        9 | Aditya Mehta |       1 |          94.00 |
|       18 | Aditya Mehta |       4 |          67.00 |
|       10 | Kavya Nair   |       1 |          81.00 |
+----------+--------------+---------+----------------+
16 rows in set (0.009 sec)
```

Figure E.3: Student Marks Retrieved Using JOIN

```
mysql> SELECT
    ->     H.Exam_ID,
    ->     A.Hall_Name,
    ->     A.Capacity
    -> FROM HALL_ALLOCATION H
    -> JOIN EXAM_HALL A
    ->     ON H.Hall_ID = A.Hall_ID;
+---------+--------------+----------+
| Exam_ID | Hall_Name    | Capacity |
+---------+--------------+----------+
|       1 | Main Hall A  |       60 |
|       4 | Main Hall A  |       60 |
|       2 | Main Hall B  |       50 |
|       3 | Main Hall B  |       50 |
|       6 | Main Hall B  |       50 |
|       3 | Seminar Hall |       40 |
|       6 | Seminar Hall |       40 |
|       5 | Block C      |       30 |
|       3 | Block A      |       30 |
|       5 | Block A      |       30 |
+---------+--------------+----------+
10 rows in set (0.012 sec)
```

Figure E.4: Examination Hall Allocation Query Output

```
mysql> SELECT
    ->     I.Exam_ID,
    ->     F.Faculty_Name
    -> FROM EXAM_INVIGILATOR I
    -> JOIN FACULTY F
    ->     ON I.Faculty_ID = F.Faculty_ID;
+---------+------------------+
| Exam_ID | Faculty_Name     |
+---------+------------------+
|       1 | Dr. Suresh Reddy |
|       4 | Dr. Suresh Reddy |
|       5 | Dr. Suresh Reddy |
|       1 | Dr. Anil Kumar   |
|       5 | Dr. Anil Kumar   |
|       2 | Dr. Priya Sharma |
|       6 | Dr. Priya Sharma |
|       2 | Prof. Rahul Verma |
|       3 | Prof. Sneha Rao  |
|       6 | Prof. Sneha Rao  |
+---------+------------------+
10 rows in set (0.016 sec)
```

Figure E.5: Invigilator Assignment Query Output